Exercise - 1

Working with DDL, DML, DCL and Key Constraints

Creation, Altering and Dropping of Tables and Inserting Rows into a Table (Use Constraints While Creating Tables). Examples Using Select Command.

Data Types :- Oracle supports following types of data types.

- **CHAR (SIZE)** :- The CHAR data type stores fixed-length character strings. When you create a table with a CHAR column, you must specify a string length (in bytes or characters) between 1 and 2000 bytes for the CHAR column width. The default is 1 byte.
- **VARCHAR2 (SIZE)** :- The VARCHAR2 data type stores variable-length character strings. When you create a table with a VARCHAR2 column, you specify a maximum string length (in bytes or characters) between 1 and 4000 bytes for the VARCHAR2 column.
- **VARCHAR (SIZE)** :- The VARCHAR data type is synonymous with the VARCHAR2 data type.
- **NUMBER (L)** :- The NUMBER datatype stores fixed and floating-point numbers. Numbers of virtually any magnitude can be stored and are guaranteed portable among different systems operating Oracle Database, up to 38 digits of precision.
- **NUMBER (L, D)** :- Numeric data with total number of digits L and number of digits D after decimal point.
- **DATE** :- The DATE datatype stores point-in-time values (dates and times) in a table. The DATE datatype stores the year (including the century), the month, the day, the hours, the minutes, and the seconds (after midnight).
- **LONG** :- Character data of variable length which stores upto 2 Gigabytes of data. (A bigger version the VARCHAR2 datatype).
- **INTEGER** :- Integer type of Data. It is actually a synonym for NUMBER(38)
- **FLOAT :-** FLOAT is a 32-bit, single-precision floating-point number datatype. Each FLOAT value requires 5 bytes, including a length byte.
- **DOUBLE** :- DOUBLE is a 64-bit, double-precision floating-point number datatype. Each DOUBLE value requires 9 bytes, including a length byte.

Data Definition Language (DDL) Commands:-

Creating Tables :-

The CREATE TABLE command is used to create the table (relation) in SQL.

CREATE TABLE TABLENAME (ATT_NAME1 DATATYPE, ATT_NAME2 DATATYPE, ATT_NAME3 DATATYPE, …..);

CREATE TABLE SAILORS (SID NUMBER (5), SNAME VARCHAR2(30), RATING NUMBER(5), AGE NUMBER(4,2));

Primary Key & Foreign Key :- Consider the Sailors relation and the constraint that no two sailors have the same SID. This type of constraint can be defined using **Primary Key**, which gives uniqueness for the value of attribute defined (Eg. SID).

Similarly, a sailor can't reserve a boat unless he/she is a valid sailor i.e. the SID of Reserves relation must available in the Sailors relation. This type of constraint can be defined using **Foreign Key**, which gives the existence of the value of attribute in one relation is depends on value in another relation.

We can use Primary Key or/and Foreign Key constraint while creating table.

Creating tables with Primary Key Constraint

CREATE TABLE TABLENAME (ATT_NAME1 DATATYPE, ATT_NAME2 DATATYPE, ATT_NAME3 DATATYPE ….., **PRIMARY KEY(ATT_NAMES)**);

CREATE TABLE SAILORS (SID NUMBER(5), SNAME VARCHAR2(30), RATING NUMBER(5), AGE NUMBER(4,2), , **PRIMARY KEY(SID)**);

Creating tables with Foreign Key Constraint

CREATE TABLE TABLENAME (ATT_NAME1 DATATYPE, ATT_NAME2 DATATYPE, ATT_NAME3 DATATYPE ….., **FOREIGN KEY (ATT_NAME) REFERENCES TABLENAME2)**);

CREATE TABLE RESERVES (SID NUMBER(5), BID NUMBER(5), DAY DATE, **FOREIGN KEY (SID) REFERENCES (SAILORS)**);

Creating table with Check Constraint: - Suppose we want to add rule for rating of the sailors - ―Rating should be between 1 to 10‖ while creating table then we can use following command.

CREATE TABLE SAILORS (SID NUMBER(5), SNAME VARCHAR2(30), RATING NUMBER(5), AGE NUMBER(4,2), , PRIMARY KEY(SID), **CHECK (RATING >=1 AND RATING <=10)**);

Here are the complete details for creating three tables using Create table command

- **CREATE TABLE SAILORS (SID NUMBER(3) PRIMARY KEY, SNAME VARCHAR2(30), RATING NUMBER(3), AGE NUMBER(4,2), CHECK (RATING >=1 AND RATING <=10))**

Output :

Table Created.

- **CREATE TABLE BOATS(BID NUMBER(3) PRIMARY KEY, BNAME VARCHAR2(20), BCOLOR VARCHAR2(20))**

Output :

Boats Table Created.

- **CREATE TABLE RESERVES (SID NUMBER(3), BID NUMBER(3), DAY DATE, PRIMARY KEY(SID,BID,DAY), FOREIGN KEY(SID) REFERENCES SAILORS, FOREIGN KEY(BID) REFERENCES BOATS)**

Output :

Reserves table Created.

- **To see all tables present in your login**

SELECT * FROM TAB;

- **To see schema of a particular table**

DESC <TABLENAME>;

- Show schema of Sailors
-

DESC SAILORS;

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
SAILORS	SID	Number	-	3	0	1	-	-	-
	SNAME	Varchar2	30	-	-	-	✓	-	-
	RATING	Number	-	3	0	-	✓	-	-
	AGE	Number	-	4	2	-	✓	-	-

- Show schema of Boats

DESC BOATS;

Output :

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
BOATS	BID	Number	-	3	0	1	-	-	-
	BNAME	Varchar2	20	-	-	-	✓	-	-
	BCOLOR	Varchar2	20	-	-	-	✓	-	-

- Show schema of Reserves

DESC RESERVES;

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
RESERVES	SID	Number	-	3	0	1	-	-	-
	BID	Number	-	3	0	2	-	-	-
	DAY	Date	7	-	-	3	-	-	-

Deleting Table :- The table along with its definition & data can be deleted using following command.

DROP TABLE <TABLENAME>;

DROP TABLE SAILORS;

Adding & Deleting the Attributes and Constraints to the Table:-

To add the attribute to an existing relation we can use ALTER TABLE Command.

ALTER TABLE <TABLENAME> ADD COLUMN ATT_NAME DATATYPE;

ALTER TABLE SAILORS ADD COLUMN SALARY NUMBER (7,2);

Output :

Display schema after modification

DESC SAILORS;

Output :

To remove the attribute from an existing relation we can use following Command.

ALTER TABLE <TABLENAME> DROP COLUMN ATT_NAME;

ALTER TABLE SAILORS DROP COLUMN SALARY;

Display schema after modification

DESC SAILORS;

Output :

To add the constraint to existing relation we can use ALTER TABLE Command.

ALTER TABLE <TABLENAME> ADD CONSTRAINT <CON_NAME> <CON_DEFINITION>;

ALTER TABLE SAILORS ADD CONSTRAINT RATE CHECK (RATING >= 1 AND RATING <=10);

Similarly we can add primary key or foreign key constraint.

To delete the constraint to existing relation we can use following Command.

DROP CONSTRAINT <CON_NAME>;

DROP CONSTRAINT RATE;

Similarly we can drop primary key or foreign key constraint.

Data Manipulation Language (DML) Commands :-

Adding data to the Table :- We can add data to table by using INSERT INTO command. While adding the data to the table we must remember the order of attributes as well as their data types as defined while creating table. The syntax is as follows.

INSERT INTO <TABLENAME> VALUES (VALUE1, VALUE2, VALUE3, …..);

INSERT INTO SAILORS VALUES (1, _Rajesh‘, 10, 30);

But sometimes while adding data we may not remember the exact order or sometimes we want to insert few values then we can use following format to add data to a table.

INSERT INTO <TABLENAME> (ATT_NAME1, ATT_NAME2, ATT_NAME3, …..) VALUES (VALUE1, VALUE2, VALUE3, …..);

INSERT INTO SAILORS (SNAME, SID, AGE, RATING) VALUES (_Rajesh‘, 1, 30, 10);

By using one of these methods we can add records or data to Sailors, Boats as well as Reserves Table.

Insert data in SAILORS Table

- INSERT INTO SAILORS VALUES(1,'VIJAY', 9, 36);
- INSERT INTO SAILORS VALUES(2,'RAJESH', 10, 25);
- INSERT INTO SAILORS VALUES(3,'MOHAN', 8, 23);
- INSERT INTO SAILORS VALUES(4,'KUMAR', 7, 28);
- INSERT INTO SAILORS VALUES(5,'SAGAR', 9, 21);
- INSERT INTO SAILORS VALUES(6,'MAHESH', 9, 36);

Insert data in BOATS Table

- INSERT INTO BOATS VALUES(1,'GANGA', 'RED');
- INSERT INTO BOATS VALUES(2,'JAMUNA', 'GREEN');
- INSERT INTO BOATS VALUES(3,'KAVERI', 'PINK');
- INSERT INTO BOATS VALUES(4,'GODAVARI', 'RED');
- INSERT INTO BOATS VALUES(5,'KRISHNA', 'BLUE');

Insert data in RESERVES Table

- INSERT INTO RESERVES VALUES(1,1,'12-FEB-2017');
- INSERT INTO RESERVES VALUES(1,2, '12-FEB-2017');
- INSERT INTO RESERVES VALUES(2,1,'13-FEB-2017‘);
- INSERT INTO RESERVES VALUES(3,2, '14-FEB-2017');
- INSERT INTO RESERVES VALUES(3,3,'14-FEB-2017');

To see the records :- To view all records present in the table.

SELECT * FROM <TABLENAME>

SELECT * FROM SAILORS;

SID	SNAME	RATING	AGE
1	VIJAY	9	36
2	RAJESH	10	25
3	MOHAN	8	23
4	KUMAR	7	28
5	SAGAR	9	21
6	MAHESH	9	36

To change particular value :- We can modify the column values in an existing row using the UPDATE command.

UPDATE <TABLENAME> SET ATT_NAME = NEW_VALUE WHERE CONDITION;

UPDATE SAILORS SET RATING = 9 WHERE SID = 2;

Output:

SID	SNAME	RATING	AGE
1	VIJAY	9	36
2	RAJESH	9	25
3	MOHAN	8	23
4	KUMAR	7	28
5	SAGAR	9	21
6	MAHESH	9	36

- To update all records without any condition.

UPDATE SAILORS SET RATING = RATING + 1;

Output:

SID	SNAME	RATING	AGE
1	VIJAY	10	36
2	RAJESH	10	25
3	MOHAN	9	23
4	KUMAR	10	28
5	SAGAR	10	21
6	MAHESH	10	36

Examples using SELECT Command

Complete Syntax of SQL Queries consist of six clauses as follows.

SELECT [DISTINCT] <attribute list>
FROM <table list>
[WHERE <condition>]
[GROUP BY <grouping attribute(s)>]
[HAVING <group condition>]
[ORDER BY <attribute list>]

Note – 1) Keywords (UPPERCASE) mentioned in [ ] are optional
2) Text in < > - data need to be provided.

- The SELECT-clause lists the attributes or functions to be retrieved
- The FROM-clause specifies all relations (or aliases) needed in the query.
- The WHERE-clause specifies the conditions for selection and join of tuples from the relations specified in the FROM-clause
- GROUP BY specifies grouping attributes
- HAVING specifies a condition for selection of groups
- ORDER BY specifies an order (sorting) for displaying the result of a query
- A query is evaluated by first applying the WHERE-clause, then GROUP BY and HAVING, and finally the SELECT-clause

Examples Using Select Command :

Show all records of Sailors

SELECT * FROM SAILORS;

Output:

SID	SNAME	RATING	AGE
1	VIJAY	10	36
2	RAJESH	10	25
3	MOHAN	9	23
4	KUMAR	10	28
5	SAGAR	10	21
6	MAHESH	10	36

Show all records of Boats

SELECT * FROM BOATS;

Output:

BID	BNAME	BCOLOR
1	GANGA	RED
2	JAMUNA	GREEN
3	KAVERI	PINK
4	GODAVARI	RED
5	KRISHNA	BLUE

- Show all records of Reserves

SELECT * FROM RESERVES;

Output:

SID	BID	DAY
1	1	12-FEB-17
1	2	12-FEB-17
2	1	13-FEB-17
3	2	14-FEB-17
3	3	14-FEB-17

- Find the names and ages of all sailors.

  SELECT DISTINCT S.SNAME, S.AGE FROM SAILORS S;

Output:

SNAME	AGE
SAGAR	21
VIJAY	36
MAHESH	36
RAJESH	25
KUMAR	28
MOHAN	23

- Find all sailors with a rating above 8.

  SELECT S.SID, S.SNAME, S.RATING, S.AGE FROM SAILORS S
  WHERE S.RATING > 8;

Same query can be written as

  SELECT * FROM SAILORS WHERE RATING > 8;

Output:

SID	SNAME	RATING	AGE
1	VIJAY	10	36
2	RAJESH	10	25
3	MOHAN	9	23
4	KUMAR	10	28
5	SAGAR	10	21
6	MAHESH	10	36

- Find sailors name with a rating above 7 & age above 25.

  SELECT SNAME FROM SAILORS S WHERE S.RATING > 7 AND S.AGE > 25;

Output:

SNAME
VIJAY
KUMAR
MAHESH

- Display all the names & colors of the boats.

  SELECT BNAME, BCOLOR FROM BOATS;

Output:

BNAME	BCOLOR
GANGA	RED
JAMUNA	GREEN
KAVERI	PINK
GODAVARI	RED
KRISHNA	BLUE

- Find all the boats with Red color.

  SELECT * FROM BOATS WHERE BCOLOR='RED';

Output :

BID	BNAME	BCOLOR
1	GANGA	RED
4	GODAVARI	RED

- Find the names of sailors who have reserved boat number 123.

  SELECT S.SNAME FROM SAILORS S, RESERVES R
  WHERE S.SID = R.SID AND R.BID = 1;

Output:

SNAME
VIJAY
RAJESH

- Find the names of sailors ' who have reserved Red boat.

  SELECT S.SNAME FROM SAILORS S, RESERVES R, BOATS B WHERE S.SID = R.SID AND R.BID = B.BID AND B.BCOLOR = 'RED' ;

Output:

SNAME
VIJAY
RAJESH

- Find the colors of boats reserved by Rajesh.

  SELECT B.BCOLOR FROM SAILORS S, RESERVES R, BOATS B WHERE S.SID = **R.SID AND R.BID = B.BID AND S.SNAME = 'RAJESH' ;**

Output:

BCOLOR
RED

- Find names of the sailors who have reserved at least one boat.

  SELECT DISTINCT S.SNAME FROM SAILORS S, RESERVES R WHERE S.SID = **R.SID;**

Output:

SNAME
VIJAY
RAJESH
MOHAN

- Find names of the sailors who have reserved two different boats.

  SELECT DISTINCT S.SNAME FROM SAILORS S, RESERVES R1, RESERVES R2 WHERE S.SID = R1.SID AND S.SID = R2.SID AND R1.BID <> R2.BID

Output:

SNAME
VIJAY
MOHAN

Data Control Language (DCL) Commands:

Data Control Language (DCL) is used to control privileges in Database. To perform any operation in the database, such as for creating tables and views - a user needs privileges.

Privileges are of two types

System: This includes permissions for creating session, tables, etc and all types of other system privileges.

Object: This includes permissions for any command or query to perform any operation on the database tables.

DCL have two commands,

GRANT: Used to provide any user access privileges or other privileges for the database.

REVOKE: Used to take back permissions from any user.

Allow a User to create session : When we create a user in SQL, it is not even allowed to login and create a session until and unless proper permissions/privileges are granted to the user. Following command can be used to grant the session creating privileges.

GRANT CREATE SESSION TO username;

Allow a User to create table : To allow a user to create tables in the database, we can use the below command,

GRANT CREATE TABLE TO username;

Grant permission to drop any table : If you want to allow user to drop any table from the database, then grant this privilege to the user,

GRANT DROP ANY TABLE TO username

To take back Permissions: If you want to take back the privileges from any user, use the REVOKE command.

REVOKE CREATE TABLE FROM username

2. Working with Queries and Nested QUERIES

Queries (along with sub Queries) using ANY, ALL, IN, EXISTS, NOTEXISTS, UNION, INTERSET, MINUS.

DISTINCT Keyword: - The DISTINCT keyword eliminates the duplicate tuples from the result records set.

Ex:- Find the Names and Ages of all sailors.

SELECT DISTINCT S.SNAME, S.AGE FROM SAILORS S;

Output:

SNAME	AGE
SAGAR	21
VIJAY	36
MAHESH	36
RAJESH	25
KUMAR	28
MOHAN	23

The answer is a set of rows, each of which is a pair (sname, age). If two or more sailors have the same name and age, the answer still contains just one pair with that name and age.

UNION, INTERSECT, EXCEPT (MINUS) :- SQL provides three set-manipulation constructs that extend the basic query form. Since the answer to a query is a multiset of rows, it is natural to consider the use of operations such as union, intersection, and difference.

SQL supports these operations under the names **UNION, INTERSECT and MINUS**.

Note : UNION, INTERSECT, and MINUS can be used on any two tables that are **Union-Compatible**, that is, have the same number of columns and the columns, taken in order, have the same data types.

UNION :- It is a set operator used as alternative to **OR** query.

Here is an example of Query using **OR.**

Ex:- Find all sids of sailors who have a rating of 10 or reserved boat number 1.

SELECT S.SID FROM SAILORS S WHERE S.RATING = 10 UNION
SELECT R.SID FROM RESERVES R WHERE R.BID = 1;

Output:

SID
1
2
4
5
6

INTERSECT :- It is a set operator used as alternative to **AND** query. Here is an example of Query using **AND.**

Ex:-

Select sid from sailors INTERSECT select sid from reserves;

Output :

SID
1
2
3

NESTED QUERIES:-

For retrieving data from the tables we have seen the simple & basic queries. These queries extract the data from one or more tables. Here we are going to see some complex & powerful queries that enables us to retrieve the data in desired manner. One of the most powerful features of SQL is nested queries. A nested query is a query that has another query embedded within it; the embedded query is called a subquery.

IN Operator :- The IN operator allows us to test whether a value is in a given set of elements; an SQL query is used to generate the set to be tested.

Ex:- Find the names of sailors who have reserved boat 103 using IN Operator.

SELECT S.SNAME FROM SAILORS S WHERE S.SID
IN (SELECT R.SID FROM RESERVES R WHERE R.BID = 1);

Output :

SNAME
VIJAY
RAJESH

NOT IN Operator: - The NOT IN is used in a opposite manner to IN.

Ex:- Find the names of sailors who have not reserved boat 103 using NOT IN Operator.

SELECT S.SNAME FROM SAILORS S WHERE S.SID
NOT IN (SELECT R.SID FROM RESERVES R WHERE R.BID = 1);

Output:

SNAME
MOHAN
KUMAR
SAGAR
MAHESH

EXISTS Operator :- This is a Correlated Nested Queries operator. The EXISTS operator is another set comparison operator, such as IN. It allows us to test whether a set is nonempty, an implicit comparison with the empty set.

Ex:- Find the names of sailors who have reserved boat number 103 using EXISTS Operator.

SELECT S.SNAME FROM SAILORS S WHERE
EXISTS (SELECT * FROM RESERVES R WHERE R.BID = 103 AND R.SID = S.SID);

Output:

SNAME
VIJAY
RAJESH

NOT EXISTS Operator :- The NOT EXISTS is used in a opposite manner to EXISTS. Ex:- Find the names of sailors who have not reserved boat number 103 using NOT EXISTS Operator.

SELECT S.SNAME FROM SAILORS S WHERE NOT EXISTS(SELECT * FROM RESERVES R WHERE R.BID = 1 AND R.SID = S.SID);

Output :

SNAME
MOHAN
KUMAR
SAGAR
MAHESH

Set-Comparison Operators:- We have already seen the set-comparison operators EXISTS, IN along with their negated versions. SQL also supports **op ANY** and **op ALL**, where **op** is one of the arithmetic comparison operators {<, <=, =, <>, >=, >}. Following are the example which illustrates the use of these Set-Comparison Operators.

ANY Operator: - It is a comparison operator. It is used to compare a value with any of element in a given set.

Ex:-

Select sname from sailors where sid = ANY (select sid from reserves where day='12-FEB-17');

Output:

SNAME
VIJAY

ALL Operator :- It is a comparison operator. It is used to compare a value with all the elements in a given set.

Ex:- Find the sailor's with the highest rating using ALL Operator.

SELECT S.SID FROM SAILORS S WHERE S.RATING >= ALL (SELECT S2.RATING FROM SAILORS S2)

Output :

SID
1
2
4
5
6